

Gauss-Jordan y Gauss-Seidel

Omar David Hernández Herrera



INDICE

Contenido

REPORTE EXAMEN	2
1.-Comparación de algoritmo	2
1.1.-Conceptos.....	2
1.2.-Ecuaciones y Procesos	3
2.-Comparación de Código	3

REPORTE EXAMEN

Comparar estos dos métodos es fundamental ya que representan dos filosofías distintas para resolver sistemas de ecuaciones lineales.

En este reporte se comparan sus algoritmos y códigos en Python, y compararemos que hace cada uno en caso de converger o no hacerlo.

1.-Comparación de algoritmo

1.1.-Conceptos

CARACTERISTICAS	GAUSS-JORDAN	GAUSS-SEIDEL
TIPO DE METODO	Directo: Encuentra la solución exacta en un numero finito de pasos	Iterativo: Se aproxima a la solución mediante repeticiones
OBJETIVO DEL PROCESO	Transforma la matriz de coeficientes en una matriz identidad	Busca que el error entre iteraciones sea menos a una tolerancia
OPERACIÓN PRINCIPÁL	Eliminación hacia adelante y hacia atrás simultáneamente	Despeje de cada variable x, y sustitución inmediata
RESULTADOS	La solución aparece directamente en el vector de términos independientes	Una aproximación que converge a la solución real

1.2.-Ecuaciones y Procesos

Gauss-Jordan

El proceso se basa en operaciones elementales de fila para reducir la matriz. No requiere sustitución hacia atrás porque cada ecuación se simplifica hasta dejar una sola incógnita

Gauss-Seidel

Se basa en la descomposición de la matriz en una matriz triangular inferior y una superior

2.-Comparación de Código

GAUSS-JORDAN

Definición de la función

```
def gauss_jordan(matriz):  
    n = len(matriz)  
    for i in range(n):  
        # 1. Verificar que el pivote no sea cero  
        if matriz[i][i] == 0:  
            for j in range(i + 1, n):  
                if matriz[j][i] != 0:  
                    matriz[i], matriz[j] = matriz[j], matriz[i]  
                    break
```

GAUSS-SEIDEL

```
import random  
import time  
import math  
def gseid(a, b, n, x, imax, es, lmbda):  
    # Paso 1: Normalización  
    for i in range(n):  
        dummy = a[i][i]  
        for j in range(n):  
            a[i][j] = a[i][j] / dummy
```

`b[i] = b[i] / dummy`

Gauss-Jordan: El `for i in range(n)` nos dice que el código va directo al grano. Sabemos exactamente cuántas vueltas va a dar: una por cada variable.

Gauss-Seidel: Aquí la función recibe parámetros como `imax` (iteraciones máximas) y `es` (error esperado). Esto significa que el código no sabe cuántas vueltas va a dar.

GAUSS-JORDAN

`# 2. Hacer que el pivote sea 1`

`pivote = matriz[i][i]`

`for k in range(len(matriz[i])):`

`matriz[i][k] /= pivote`

GAUSS-SEIDEL

`# Paso 2: Aproximación inicial`

`for i in range(n):`

`suma = b[i]`

`for j in range(n):`

`if i != j:`

`suma -= a[i][j] * x[j]`

`x[i] = suma`

Gauss-Jordan: El código busca el **pivote** y divide todo el renglón entre él (`matriz[i][k] /= pivote`). Su meta es que ese número se convierta en un **1**.

Gauss-Seidel: No divide todo el renglón de golpe. Lo que hace es un **despeje**. Toma el resultado (`b[i]`) y le va restando lo que valen las otras variables (`suma -= a[i][j] * x[j]`)

GAUSS-JORDAN

`# 3. Hacer ceros en las demás filas`

`for j in range(n):`

`if j != i:`

`factor = matriz[j][i]`

```

        for k in range(len(matriz[j])):
            matriz[j][k] -= factor * matriz[i][k]

    return matriz

resultado = gauss_jordan(A)

print("Matriz en forma reducida:")

for fila in resultado:
    print(fila)

```

GAUSS-SEIDEL

```

# Paso 3: Iteraciones

itera = 1

while True:
    centinela = 1
    for i in range(n):
        old = x[i]
        suma = b[i]
        for j in range(n):
            if i != j:
                suma -= a[i][j] * x[j]
        x[i] = lmbda * suma + (1.0 - lmbda) * old
        if x[i] != 0:
            ea = abs((x[i] - old) / x[i]) * 100
            if ea > es:
                centinela = 0

    itera += 1

    if centinela == 1 or itera >= imax:
        break

    return x, itera

# ----- GENERAR MATRIZ DOMINANTE -----

def generar_matriz(n):
    A = [[random.uniform(-10,10) for j in range(n)] for i in range(n)]
    B = [random.uniform(-10,10) for i in range(n)]

    # asegurar dominancia diagonal

```

```

for i in range(n):
    suma = sum(abs(A[i][j]) for j in range(n) if j != i)
    A[i][i] = suma + random.uniform(5,15)

return A, B

# ----- TAMAÑO DEL SISTEMA -----
n = 20 # cambiar a 10 si quieres
A, B = generar_matriz(n)
X_inicial = [0.0]*n

# ----- MEDICIÓN DE TIEMPO -----
inicio = time.perf_counter()
resultado, iteraciones = gseid(A, B, n, X_inicial, 500, 0.001, 1.0)
fin = time.perf_counter()
tiempo = fin - inicio

# ----- RESULTADOS -----
print("Solución:")

for i in range(n):
    print("fx{i+1} = {resultado[i]}")

print("\nIteraciones:", iteraciones)
print("Tiempo de ejecución:", tiempo, "segundos")

```

Gauss-Jordan: El algoritmo finaliza tras completar la eliminación sistemática de todas las filas y columnas, transformando la matriz original en su forma escalonada reducida.

Gauss-Seidel: Emplea una estructura de **control estocástica o condicional** (while True). La finalización no depende de un número fijo de pasos, sino de un criterio de convergencia.